

UNIVERSIDADE DO VALE DO ITAJAÍ – UNIVALI
CURSO DE [PREENCHER]

MATHEUS THUROW
RODRIGO NICOLA
EDUARDO

PROCESSAMENTO CONCORRENTE DE REQUISIÇÕES A UM BANCO DE
DADOS SIMULADO NO WINDOWS

[CIDADE]
2026

MATHEUS THUROW

RODRIGO NICOLA

EDUARDO

**PROCESSAMENTO CONCORRENTE DE REQUISIÇÕES A UM BANCO DE
DADOS SIMULADO NO WINDOWS**

Relatório apresentado à disciplina de Sistemas
Operacionais do curso de [preencher], da
Universidade do Vale do Itajaí, como parte da
Avaliação M1 de 2026/2.

Professor: [preencher].

[CIDADE]
2026

RESUMO

Este trabalho apresenta um sistema em C++ que simula o processamento de requisições a um banco de dados no Windows. A solução utiliza processos cliente e servidor separados, comunicação por named pipe, um pool de threads e mutexes para proteger as estruturas compartilhadas. Os resultados são registrados em log e retornados ao cliente. Foram implementadas as operações INSERT, SELECT, UPDATE e DELETE sobre um vetor de registros. A validação funcional verificou essas operações, o tratamento de entradas inválidas e a disputa por um mesmo identificador. A avaliação de desempenho comparou pools de uma, duas, quatro e oito threads, com quatro clientes e lotes de 100, 1.000 e 5.000 inserções, em cinco repetições por cenário. As 60 medições finais totalizaram 122.000 inserções verificadas sem perdas, duplicações ou respostas inesperadas. Para 5.000 inserções, as medianas foram 0,3365 s com uma thread e 0,2731 s com quatro. Os resultados demonstram funcionamento correto nos cenários testados, com redução de 18,9% nesse lote, mas sem ganho consistente em todos os cenários. A serialização do banco, a comunicação e a escrita dos logs são fatores compatíveis com esse comportamento.

Palavras-chave: comunicação entre processos; threads; exclusão mútua; banco de dados simulado; Windows.

ABSTRACT

This report presents a C++ system that simulates database request processing on Windows. The solution uses separate client and server processes, named-pipe communication, a thread pool, and mutexes to protect shared data structures. Results are logged and returned to the client. INSERT, SELECT, UPDATE, and DELETE operations were implemented over a vector of records. Functional tests checked these operations, invalid inputs, and concurrent attempts to insert the same identifier. Performance experiments compared pools of one, two, four, and eight threads, with four clients and batches of 100, 1,000, and 5,000 insertions, using five repetitions per scenario. The 60 final measurements covered 122,000 insertions with no missing records, duplicates, or unexpected responses in the verified logs. For 5,000 insertions, median batch times were 0.3365 seconds with one thread and 0.2731 seconds with four. The tested scenarios behaved correctly, but increasing the pool did not yield a consistent performance improvement. Serialized database access, communication, and logging are factors consistent with this result. The measurements describe the complete batch, including synchronization and shutdown, rather than the latency of an individual request.

Keywords: interprocess communication; threads; mutual exclusion; simulated database; Windows.

SUMÁRIO

1 INTRODUÇÃO	5
1.1 Enunciado e requisitos	5
1.2 Processos threads e sincronização	6
2 ARQUITETURA E IMPLEMENTAÇÃO	7
2.1 Trechos principais da implementação	8
3 METODOLOGIA EXPERIMENTAL	9
4 RESULTADOS E DISCUSSÃO.....	10
4.1 Distribuição dos tempos	11
4.2 Validação funcional e utilização do pool.....	12
4.3 Discussão e limitações	12
5 CONCLUSÃO	13
REFERÊNCIAS	14
APÊNDICE A – INSTRUÇÕES DE EXECUÇÃO	15

1 INTRODUÇÃO

O atendimento de requisições por múltiplos clientes exige coordenação entre unidades de execução. Quando diferentes threads acessam uma mesma tabela, operações como verificar a existência de um identificador e inserir um registro precisam ocorrer de forma protegida para evitar condições de corrida.

O objetivo deste projeto é aplicar os conceitos de IPC, threads e sincronização em uma implementação simples e executável, conforme a proposta da Avaliação M1 (1). Além de verificar o comportamento funcional, o trabalho compara o tempo de atendimento de lotes com diferentes tamanhos de pool. O estudo distingue o uso correto da concorrência de um eventual ganho de desempenho.

1.1 Enunciado e requisitos

A proposta consiste em simular um gerenciador de requisições a um banco de dados. Um processo cliente envia comandos por IPC real a um processo servidor, que mantém um pool de threads para atendê-los. As threads compartilham uma estrutura que representa o banco e devem utilizar mutex ou semáforo para preservar sua integridade. Devem ser suportadas inserção, consulta, atualização e exclusão de registros, com respostas por outro canal IPC ou por arquivo de log (1).

A implementação utiliza cliente.exe e servidor.exe. O banco é um vetor em memória, com ID e nome. As respostas são gravadas em servidor.log, exibidas no servidor e devolvidas ao cliente pelo named pipe. O log atende à alternativa prevista no enunciado; o retorno ao cliente facilita o uso. A sintaxe dos comandos é textual e reduzida.

Quadro 1 – Correspondência entre requisitos e implementação

Requisito	Implementação
Processos separados	cliente.exe e servidor.exe, cada um com sua função main.
IPC real	Named pipe do Windows, criado com CreateNamedPipeA.
Pool de threads	Threads criadas com CreateThread dentro de servidor.cpp.
Proteção dos dados	Mutex do banco criado com CreateMutexA; fila e log têm mutexes próprios.
Sinalização de trabalho	Semáforo criado com CreateSemaphoreA.
Operações e respostas	INSERT, SELECT, UPDATE e DELETE; resultados no log e no cliente via IPC.

Fonte: elaboração dos autores com base na implementação e nos dados experimentais (2026).

1.2 Processos threads e sincronização

Um processo reúne os recursos necessários à execução de um programa. As threads de um mesmo processo compartilham seu espaço de endereçamento e seus recursos (2). Neste projeto, a memória do banco pertence somente ao servidor; o cliente não a acessa diretamente e envia suas solicitações pelo named pipe. Esse mecanismo permite comunicação entre processos independentes (3).

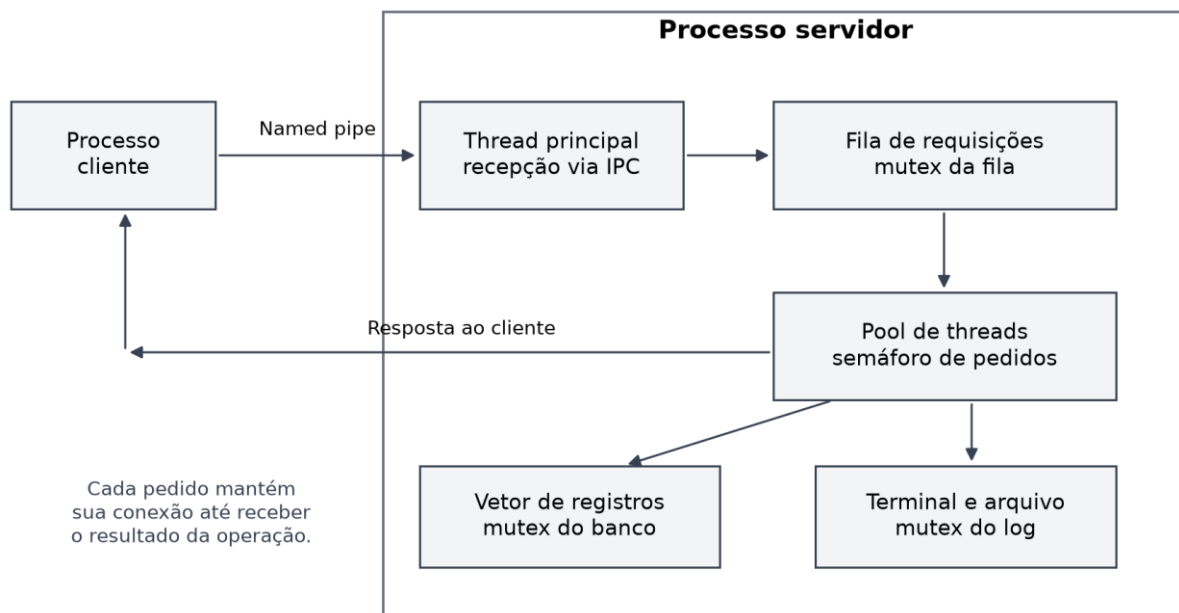
O mutex concede propriedade exclusiva a uma thread por vez (4). Já o semáforo mantém uma contagem que é reduzida quando uma espera é satisfeita e aumentada quando há liberação (5). Na solução, o mutex protege a tabela, enquanto o semáforo sinaliza a existência de pedidos na fila. Essa separação evita que as threads consultem continuamente uma fila vazia.

Foram escolhidas APIs nativas do Windows equivalentes em finalidade às primitivas citadas na proposta: `CreateThread` para criação, `CreateMutexA` para exclusão mútua e `CreateSemaphoreA` para sinalização. A implementação preserva os conceitos de threads e sincronização, mas não utiliza a interface POSIX. O pool é construído pelo programa, sem utilizar o serviço de thread pool pronto do Windows.

2 ARQUITETURA E IMPLEMENTAÇÃO

A thread principal recebe o comando e enfileira um Pedido com texto e handle do pipe. Outra instância recebe novos clientes. O semáforo acorda uma thread do pool, que retira o pedido, executa a operação sob o mutex do banco e registra o resultado sob o mutex do log. Fora desses mutexes, devolve a resposta ao cliente e fecha a conexão (Figura 1).

Figura 1 – Organização dos processos e das estruturas compartilhadas



Fonte: elaboração dos autores com base na implementação e nos dados experimentais (2026).

O cliente espera o resultado completo antes de enviar outro comando. Pedidos do mesmo cliente terminam em sequência; entre clientes distintos, o pool não garante a ordem de conclusão. O retorno não substitui o log.

IDs são inteiros não negativos; nomes admitem espaços e até 49 bytes. PARAR conclui os pedidos recebidos antes de encerrar. O banco é volátil. CLS limpa a tela do cliente, mantendo as instruções fixas no topo em terminais compatíveis.

2.1 Trechos principais da implementação

O pool é criado uma vez em servidor.cpp e reutilizado. O servidor aceita de uma a 64 threads, com quatro como padrão. Os números passados como argumentos permanecem válidos até o término do pool.

Código 1 – Criação do pool em servidor.cpp

```
for (int i = 0; i < quantidade_threads; ++i) {
    numeros[i] = i + 1;
    threads[i] = CreateThread(nullptr, 0, atender,
                             &numeros[i], 0, nullptr);
    if (!threads[i]) break;
    ++criadas;
}
```

CreateThread recebe a função que a thread executará e seu argumento (6). Na rotina atender, o semáforo controla a espera por trabalho. O bloco delimitado abaixo protege somente a retirada da fila; a execução da operação ocorre depois de sua liberação.

Código 2 – Espera e retirada de um pedido na rotina atender

```
WaitForSingleObject(tem_requisicao, INFINITE);
Pedido pedido;
{
    GuardaMutex guarda(mutex_fila);
    if (fila.empty() && encerrando) break;
    if (fila.empty()) continue;
    pedido = fila.front();
    fila.pop();
}
std::string resposta = executar_requisicao(pedido.texto);
```

GuardaMutex adquire o mutex no construtor e o libera no destrutor. Assim, a saída do bloco libera a proteção. Em banco.cpp, o mesmo recurso cobre toda a operação, incluindo a busca do ID. SELECT também adquire o mutex, pois uma escrita simultânea poderia modificar o vetor durante a leitura.

Código 3 – Proteção das operações em banco.cpp

```
std::string executar_requisicao(const std::string& texto) {
    GuardaMutex guarda(mutex_banco);
    return executar(texto);
}
```

Depois do log, responder_e_fechar(pedido.pipe, resposta) devolve o resultado. Com PARAR, WaitForMultipleObjects aguarda o pool concluir os pedidos; só então o servidor responde ao pedido de encerramento.

3 METODOLOGIA EXPERIMENTAL

Quadro 2 – Ambiente das simulações de 20 de setembro de 2026

Item	Configuração
Sistema	Windows 11, versão 10.0.26200
Processador	Intel Core i5-12450HX; 8 núcleos e 12 processadores lógicos
Memória	15,73 GiB utilizáveis pelo sistema operacional
Compilação	MinGW GCC 6.3.0; C++11; -static; sem opção explícita -O
Automação	Python 3.12.14; script testes/simular.py
Cenários	1, 2, 4 e 8 threads; 100, 1.000 e 5.000 inserções
Clientes e repetições	4 clientes; 5 repetições por combinação; 60 medições

Fonte: hardware.json (mesma máquina caracterizada em 19/09) e dados.json da nova rodada (2026).

Cada execução iniciou um servidor com banco vazio. Os IDs foram distintos, de zero a $N-1$, distribuídos entre quatro clientes. O tamanho do lote corresponde ao total enviado pelos quatro processos. A carga de desempenho contém somente INSERT; as demais operações foram verificadas separadamente no teste funcional. Não foram adicionados atrasos artificiais para favorecer uma configuração.

Houve um aquecimento de 1.000 inserções para cada pool. Os 60 ensaios foram embaralhados com semente 20260919; nenhum foi excluído. Usou-se uma cópia com apenas o nome do pipe alterado, evitando conflito com o servidor manual aberto. METODO.md e fontes_medidos registram essa condição. As rodadas anteriores não compõem estas estatísticas.

O tempo foi medido por `time.perf_counter()`, do despacho dos envios à saída do servidor após PARAR. Inclui IPC de ida e volta, operações, sincronização, logs, coordenação Python e encerramento. Os processos foram lançados antes do cronômetro; parte da inicialização pode estar incluída. A validação do log ocorreu após a medição.

Cada cliente aguardou o resultado antes de enviar o próximo pedido. O log ficou ativo, com flush. A saída do servidor foi redirecionada e a dos clientes, descartada; o cabeçalho interativo ficou desativado. Medem-se lotes completos, não latências individuais. Usaram-se mediana, média, desvio padrão amostral e faixa observada. $Vazão = N/mediana$; $aceleração = mediana \text{ de uma thread } / mediana \text{ do cenário}$.

4 RESULTADOS E DISCUSSÃO

As 60 medições finais totalizaram 122.000 inserções. Todas apresentaram a quantidade esperada de respostas, IDs completos e distintos, correspondência entre ID e nome e códigos de saída iguais a zero. Os quatro aquecimentos adicionais, com 4.000 inserções, não estão incluídos na Tabela 1.

Tabela 1 – Tempos e vazão por cenário com cinco repetições

Lote	Threads	Mediana (s)	Média ± DP (s)	Mín. a máx. (s)	Vazão (req/s)	Acel. (vezes)
100	1	0,0192	0,0285 ± 0,0202	0,0146 a 0,0625	5.216	1,00
100	2	0,0301	0,0417 ± 0,0178	0,0276 a 0,0635	3.325	0,64
100	4	0,0207	0,0221 ± 0,0064	0,0154 a 0,0294	4.821	0,92
100	8	0,0201	0,0295 ± 0,0202	0,0167 a 0,0650	4.963	0,95
1.000	1	0,0632	0,0691 ± 0,0236	0,0472 a 0,1063	15.820	1,00
1.000	2	0,0554	0,0718 ± 0,0339	0,0514 a 0,1313	18.064	1,14
1.000	4	0,0788	0,0965 ± 0,0414	0,0641 a 0,1662	12.686	0,80
1.000	8	0,1255	0,1174 ± 0,0426	0,0507 a 0,1535	7.967	0,50
5.000	1	0,3365	0,3133 ± 0,0516	0,2479 a 0,3583	14.857	1,00
5.000	2	0,3330	0,3412 ± 0,0606	0,2541 a 0,4157	15.013	1,01
5.000	4	0,2731	0,2805 ± 0,0272	0,2501 a 0,3151	18.309	1,23
5.000	8	0,3683	0,3720 ± 0,0766	0,2733 a 0,4745	13.577	0,91

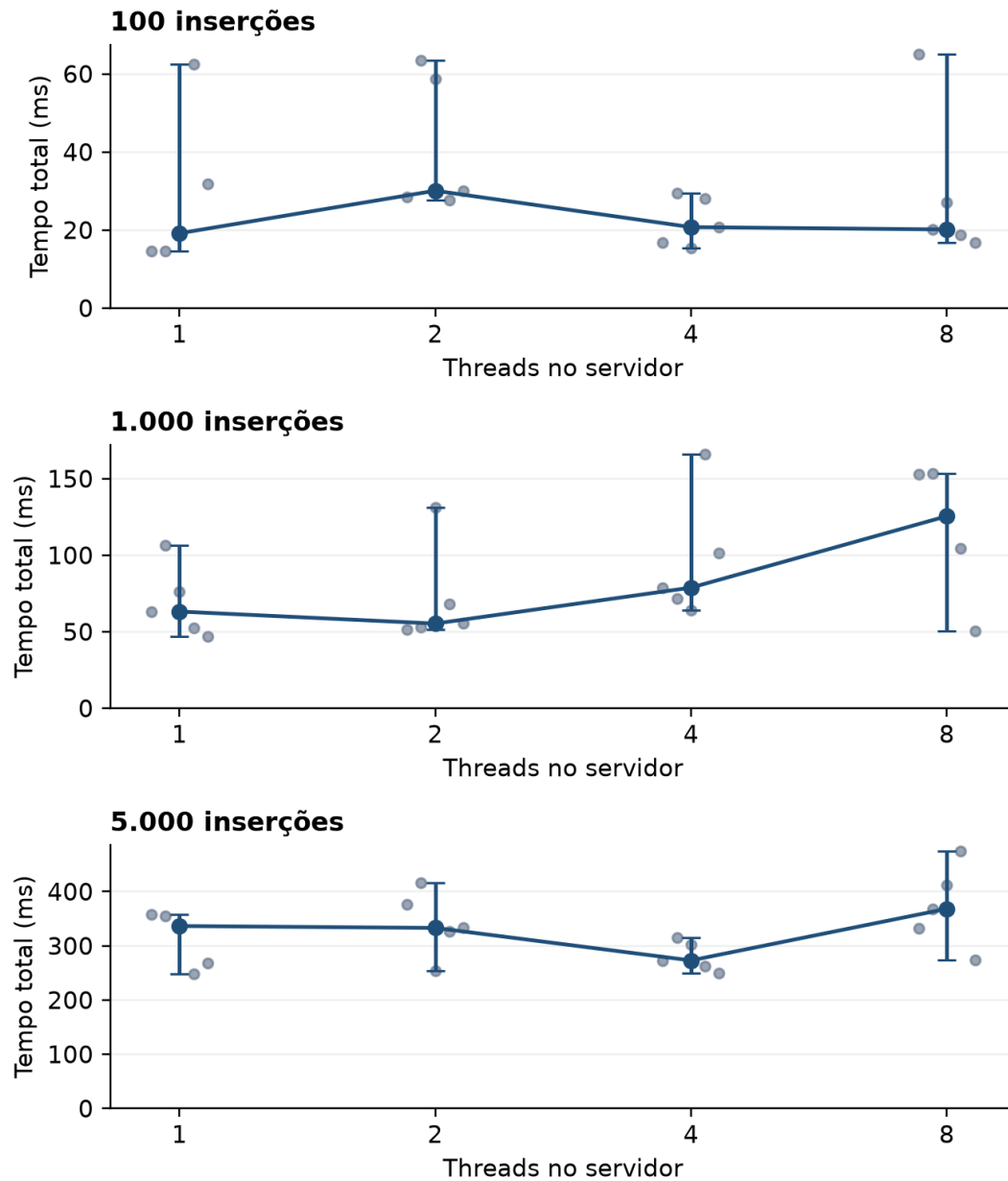
Fonte: dados.json e resumo.json da rodada simulacao_20260920_114551 (2026).

DP representa o desvio padrão amostral. Aceleração abaixo de 1 indica que a mediana foi maior que a obtida com uma thread para o mesmo lote. A faixa mínima e máxima descreve a dispersão observada e não corresponde a um intervalo de confiança.

No lote de 5.000 inserções, uma thread teve mediana de 0,3365 s e vazão de 14.857 requisições/s. Com quatro threads, a mediana foi 0,2731 s, 18,9% menor, e a vazão foi 18.309 requisições/s. A aceleração observada foi 1,23 vez. As faixas se sobrepõem; a comparação não estabelece uma vantagem universal.

4.1 Distribuição dos tempos

Figura 2 – Tempos dos lotes conforme o número de threads



Fonte: elaboração dos autores a partir das 60 medições finais (2026).

A Figura 2 apresenta as cinco execuções de cada cenário. A linha representa a mediana e as barras indicam mínimo e máximo; não são intervalos de confiança. As escalas verticais são independentes. O tempo inclui IPC, logs e encerramento.

4.2 Validação funcional e utilização do pool

O teste funcional verificou CRUD e erros de entrada no cliente e no log, além de uma sequência de operações dependentes. Na disputa de 12 clientes pelo mesmo ID, uma inserção foi aceita e 11 rejeitadas. Também conferiu respostas de 24 clientes com IDs distintos, com até 12 processos simultâneos, 20 inserções seguidas de PARAR e falha de conexão após o encerramento.

Nos cinco lotes de 5.000 inserções com quatro threads, as contagens foram T1: 6.260, T2: 6.248, T3: 6.239, T4: 6.253. O total foi 25.000. Todas as threads participaram, mas isso não demonstra execução simultânea dentro do banco, pois sua seção crítica é exclusiva.

4.3 Discussão e limitações

As menores medianas ocorreram com uma thread para 100 inserções, duas para 1.000 e quatro para 5.000. Oito threads não melhoraram a mediana frente a uma thread em nenhum volume. Há dispersão e sobreposição entre faixas; não foi realizado teste de significância.

Não houve redução monotônica do tempo ao ampliar o pool. O acesso ao vetor e a busca sequencial do ID ocorrem sob um único mutex. A recepção é sequencial, mas os pedidos mantêm conexões independentes até receber a resposta da thread. O log também tem mutex e flush. Esses custos coexistem com o atendimento concorrente; o experimento não isolou a contribuição de cada etapa.

As medições têm limites: cinco repetições por cenário, lotes curtos, uma única máquina, banco inicialmente vazio, carga apenas de inserção e compilação sem otimização explícita. O computador não foi isolado de toda atividade de fundo. Não foram coletados uso de CPU, consumo de memória, percentis de latência ou intervalos de confiança. Os resultados não devem ser generalizados para bancos persistentes ou cargas intensivas de cálculo.

O banco não é persistente e a fila não tem limite. O protocolo não oferece transações compostas nem ordenação entre clientes distintos. Cada cliente aguarda sua própria resposta. Leituras e escritas IPC não têm timeout; um cliente que deixe de ler pode ocupar uma thread. O projeto pressupõe os clientes locais fornecidos.

5 CONCLUSÃO

O sistema implementou IPC real, um pool de threads e proteção das estruturas compartilhadas no Windows. O servidor mantém o log previsto no enunciado e devolve os resultados ao cliente. Os testes funcionais e as simulações verificaram os cenários avaliados, sem perdas ou duplicações nas 122.000 inserções medidas.

A concorrência correta não garante aceleração em toda carga. Nesta amostra, quatro threads reduziram a mediana em 18,9% no lote de 5.000 inserções, mas as diferenças variaram com o volume e apresentaram dispersão. Novos estudos podem ampliar lotes e repetições e incluir consultas e atualizações. Alterações no banco ou no log exigem novas medições.

REFERÊNCIAS

- 1 UNIVERSIDADE DO VALE DO ITAJAÍ. **Avaliação M1: IPC, Threads e Paralelismo**. [S. l.]: Univali, 2026. Material da disciplina de Sistemas Operacionais, 2026/2. Arquivo Trabalho M1_2026_2_propost.pdf.
- 2 MICROSOFT. **About Processes and Threads**. Microsoft Learn, 14 jul. 2025. Disponível em: <https://learn.microsoft.com/en-us/windows/win32/procthread/about-processes-and-threads>. Acesso em: 19 set. 2026.
- 3 MICROSOFT. **Named Pipes**. Microsoft Learn, 7 jan. 2021. Disponível em: <https://learn.microsoft.com/en-us/windows/win32/ipc/named-pipes>. Acesso em: 19 set. 2026.
- 4 MICROSOFT. **Mutex Objects**. Microsoft Learn, 7 jan. 2021. Disponível em: <https://learn.microsoft.com/en-us/windows/win32/sync/mutex-objects>. Acesso em: 19 set. 2026.
- 5 MICROSOFT. **Semaphore Objects**. Microsoft Learn, 7 jan. 2021. Disponível em: <https://learn.microsoft.com/en-us/windows/win32/sync/semaphore-objects>. Acesso em: 19 set. 2026.
- 6 MICROSOFT. **Creating Threads**. Microsoft Learn, 17 jul. 2026. Disponível em: <https://learn.microsoft.com/en-us/windows/win32/procthread/creating-threads>. Acesso em: 19 set. 2026.

APÊNDICE A – INSTRUÇÕES DE EXECUÇÃO

O pacote inclui fontes, executáveis, testes e resultados. No Git, os executáveis são ignorados e devem ser compilados. Com g++ no PATH, compilar.bat gera cliente.exe e servidor.exe. Execute os comandos abaixo; o cliente mostra as respostas, e o servidor também as grava em servidor.log.

```
.\compilar.bat
.\servidor.exe 4      # primeiro terminal
.\cliente.exe        # segundo terminal
python testes/testar.py
python testes/simular.py
```

Feche o servidor manual antes dos testes: cada script inicia o próprio servidor. A rodada analisada está em resultados/simulacao_20260920_114551. dados.json preserva tempos e hashes; resumo.json reúne estatísticas. METODO.md explica o canal isolado, e fontes_medidos contém a cópia testada. CLS limpa o cliente; SAIR fecha apenas esse processo.

Quadro 3 – Comandos aceitos pelo sistema

Comando	Exemplo	Comportamento
INSERT	INSERT 1 Ana Silva	Insere um registro; rejeita ID existente.
SELECT	SELECT 1	Consulta o registro pelo ID.
UPDATE	UPDATE 1 Maria Silva	Altera o nome de um registro existente.
DELETE	DELETE 1	Remove o registro pelo ID.
SAIR / PARAR	SAIR ou PARAR	Fecha o cliente ou encerra o servidor, respectivamente.

Fonte: elaboração dos autores com base na implementação e nos dados experimentais (2026).

Repositório público do projeto: [inserir link após a publicação].